

Introduction to Functions

A function is a reusable block of code that performs a specific task.

Why Functions?

1 Without Functions:

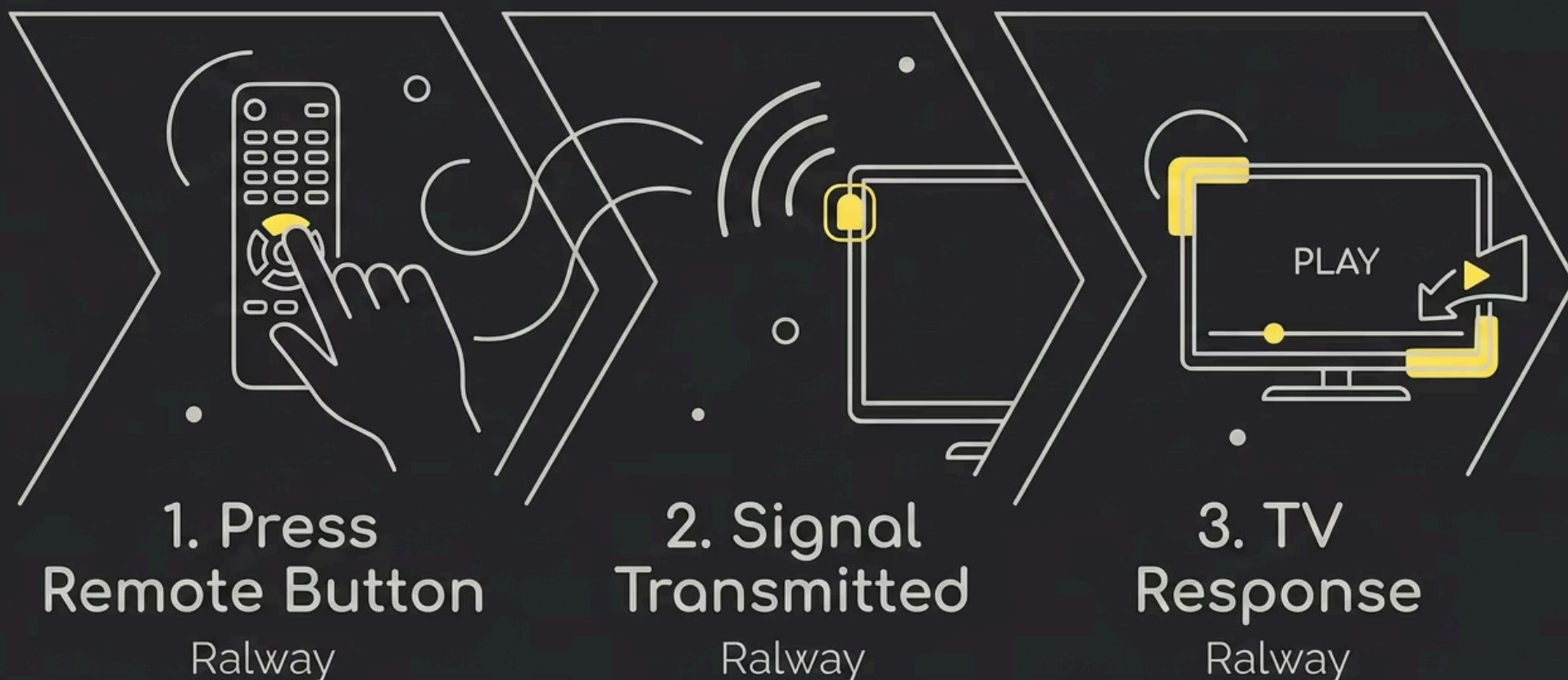
```
console.log("Welcome");  
console.log("Welcome");  
console.log("Welcome");
```

Repeating code again and again.

2 With Functions:

```
function greet(){  
  console.log("Welcome");  
}  
greet();
```

Real World Example



Benefits of Functions

Reusable code

Cleaner programs

Easier maintenance

Less repetition

Tip

Ask students: If you need to calculate total marks 100 times, will you write the same code 100 times?

Creating Functions

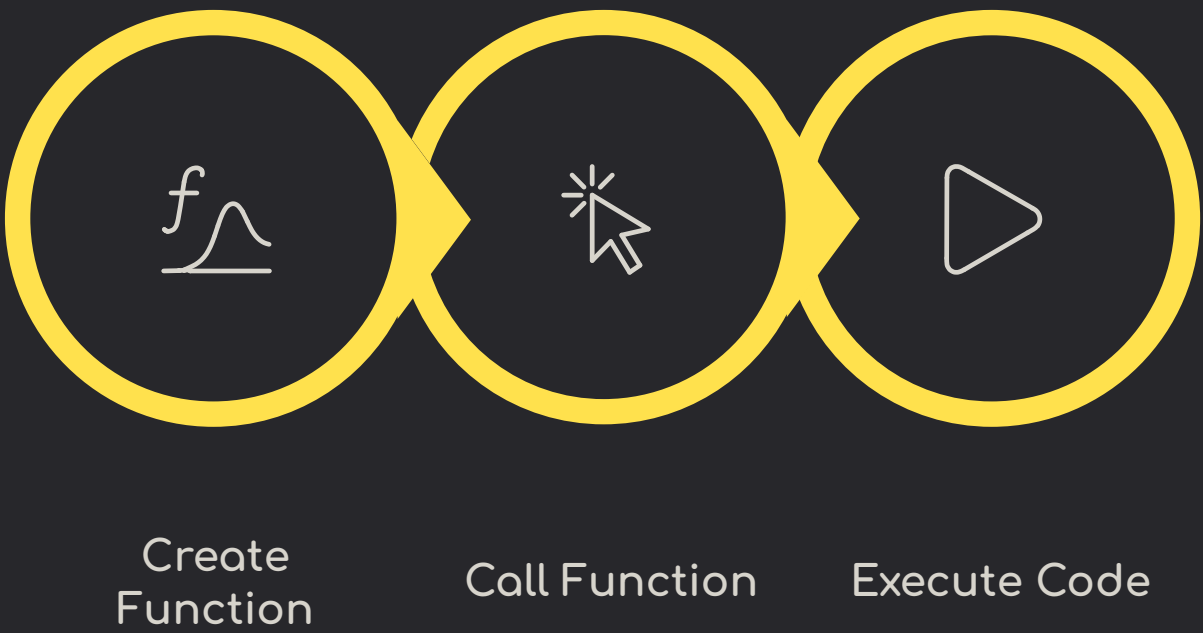
Function Syntax

```
function functionName(){  
  // code  
}
```

Example

```
function welcome(){  
  console.log("Welcome Students");  
}  
welcome();
```

Function Flow



Multiple Calls

```
welcome();  
welcome();  
welcome();
```

Output:

```
Welcome Students  
Welcome Students  
Welcome Students
```

Real World Usage

Functions are used in:



Login systems



Payment systems



Search features



Calculator apps

 Teaching Tip

Create a function that prints student's name.

Function Parameters

What are Parameters?

Parameters allow functions to receive data.

Example Without Parameter

```
function greet(){  
  console.log("Hello");  
}
```

Always prints same output.

Example With Parameter

```
function greet(name){  
  console.log("Hello " + name);  
}  
greet("Rahul");
```

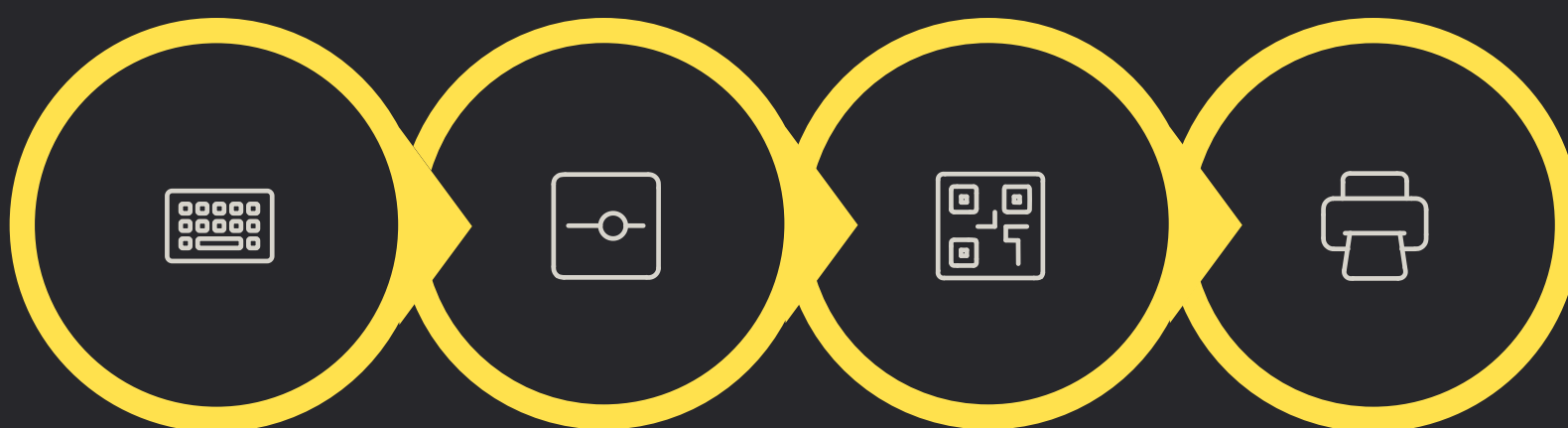
Output

Hello Rahul

Multiple Parameters

```
function student(name, age){  
  console.log(name, age);  
}  
student("Rahul", 21);
```

Visual Understanding



Input

Parameter

Function

Output

Teaching Tip

Let students pass their own name and age.

Return Keyword

What is return?

`return` sends a value back from a function.

Example

```
function add(a, b){  
  return a + b;  
}  
let result = add(10, 20);  
console.log(result);
```

Output

30

Flow

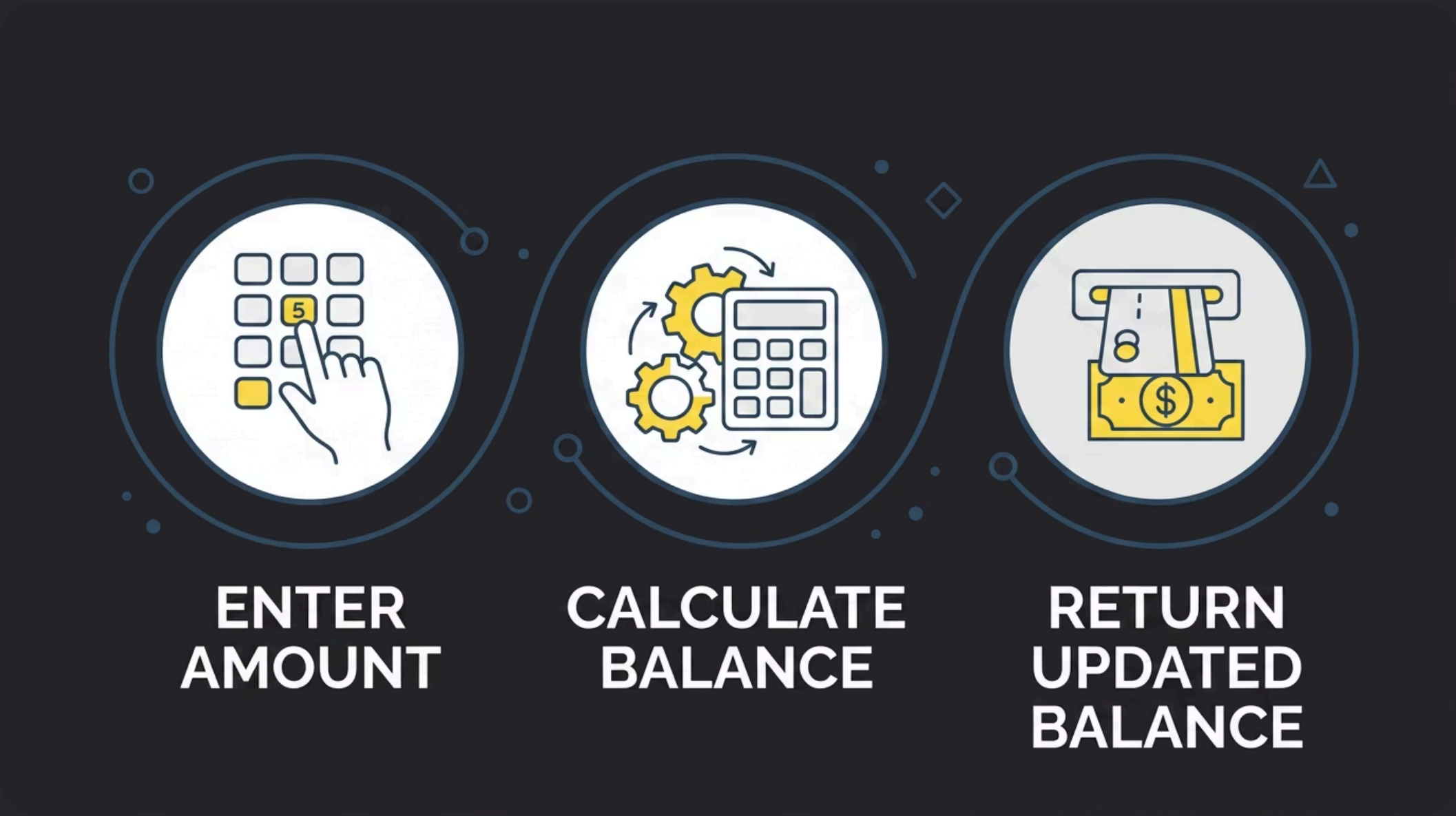


Why Use return?

Without return:
Only Displays Result

With return:
Stores Result
Reuse Result

Real World Example



Teaching Tip

Show difference between `console.log()` and `return`.

Calculator Using Functions

Example Calculator

```
function add(a, b){  
  return a + b;  
}  
console.log(add(10, 5));
```

Subtraction Function

```
function subtract(a, b){  
  return a - b;  
}
```

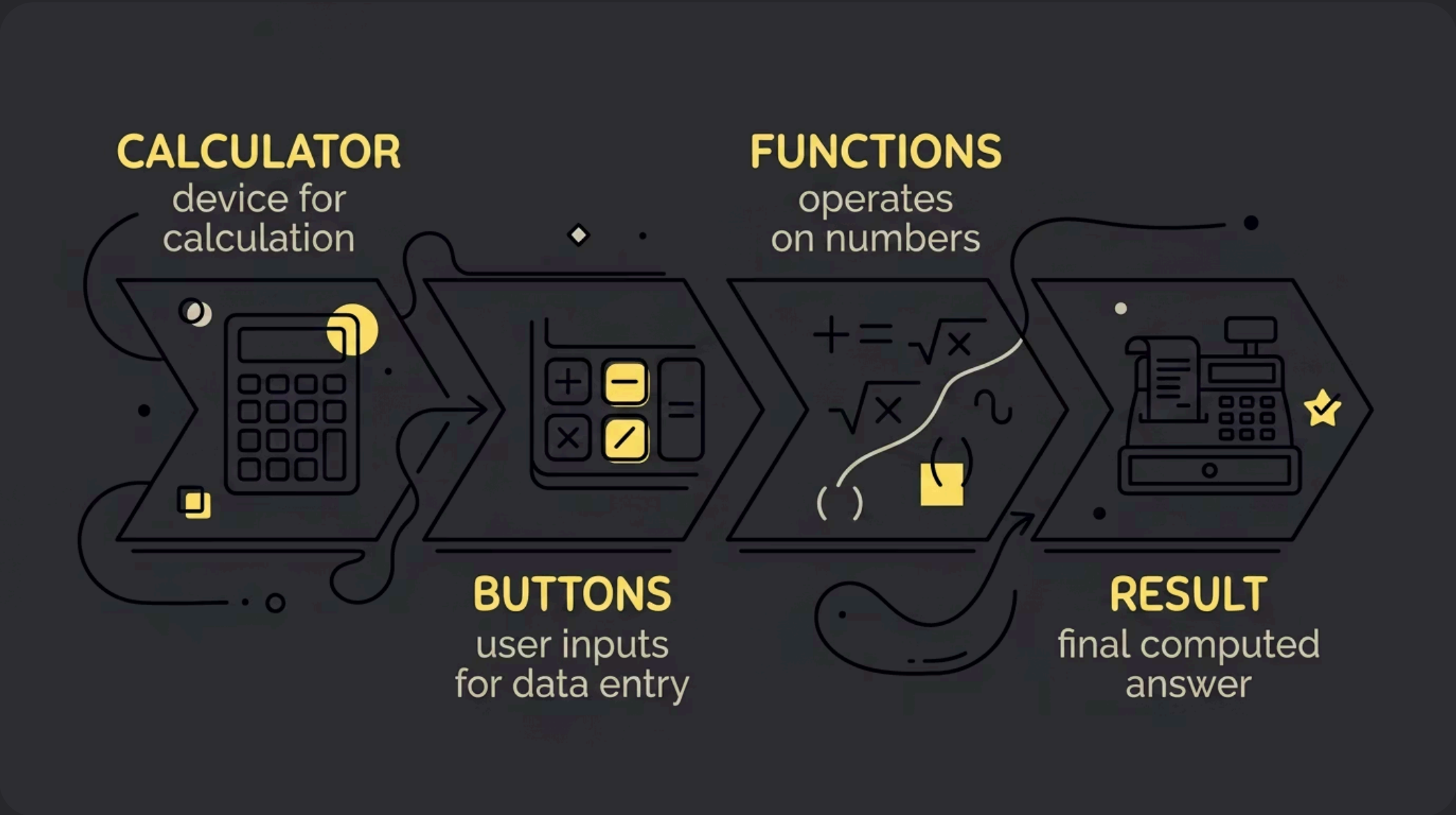
Multiplication Function

```
function multiply(a, b){  
  return a * b;  
}
```

Division Function

```
function divide(a, b){  
  return a / b;  
}
```

Real World Connection



 Teaching Tip

Build calculator step by step live.

Arrow Functions

What are Arrow Functions?

Modern and shorter way to write functions.

Traditional Function

```
function greet(){  
  console.log("Hello");  
}
```

Arrow Function

```
const greet = () => {  
  console.log("Hello");  
}
```

Arrow Function with Parameters

```
const add = (a, b) => {  
  return a + b;  
}
```

Short Form

```
const add = (a, b) => a + b;
```

Why Companies Use Arrow Functions?

- Cleaner syntax
- Used heavily in React
- Modern JavaScript standard

Teaching Tip

Mention that React developers use arrow functions daily.

Understanding Scope

What is Scope?

Scope determines where variables can be accessed.

Types of Scope



Real Life Analogy

Home WiFi Password

Known to Everyone in Home



Global Scope

Personal Diary

Accessible Only to Owner



Local Scope

Why Scope Matters?

Prevents:

- ➔ Variable conflicts
- ➔ Unexpected errors
- ➔ Data leaks

Teaching Tip

Draw scope boxes on board.

Global and Local Scope

Global Scope

Variable accessible everywhere.

```
let college = "ABC College";  
function show(){  
  console.log(college);  
}  
show();
```

Local Scope

Variable accessible only inside function.

```
function show(){  
  let student = "Rahul";  
  console.log(student);  
}  
show();
```

Error Example

```
console.log(student);
```

Output:

Reference Error

Visual Understanding

Global Variable

Accessible Everywhere

Local Variable

Accessible Inside Function Only

📌 Teaching Tip

Run both examples live.

Block Scope

What is Block Scope?

Variables declared using `let` and `const` are limited to a block.

Example

```
if(true){  
  let marks = 95;  
  console.log(marks);  
}
```

Works inside block.

Outside Block

```
console.log(marks);
```

Output:

Reference Error

Visual Representation



{ Block Scope }

Only available inside braces.

Important Note

`let` and `const` → Block Scoped

`var` → Function Scoped

Teaching Tip

Explain why `let` is safer than `var`.

Mini Task, Homework and Outcome

Mini Task

Calculator Using Functions

Create:

- Addition
- Subtraction
- Multiplication
- Division

Functions

Example

```
function add(a, b){  
  return a + b;  
}
```

Homework

BMI Calculator

Formula:

BMI = Weight / Height²

Example:

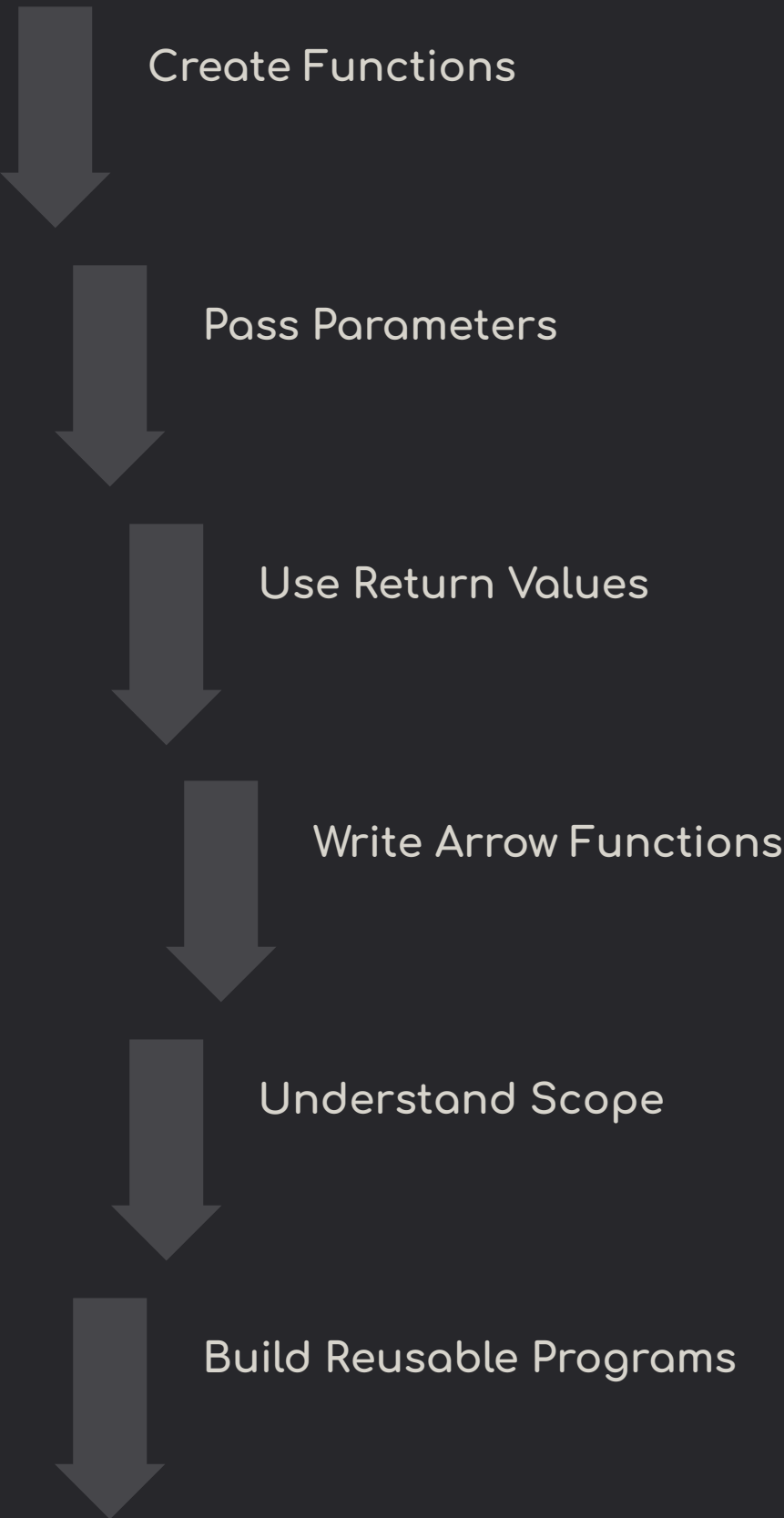
```
let weight = 70;  
let height = 1.75;
```

Expected Output:

BMI = 22.86

Final Outcome

After Day 4 students will:



 **Important Tip**

This is the most important JavaScript foundation topic before arrays, objects, DOM, and React. Spend extra time on parameters, return, and scope because students use them throughout their entire development career.